# Denticon Integration

## 1. Overview

### 1.1 Description

**Denticon Integration** connects **Denticon** (PlanetDDS dental practice management system) with **Newo Platform**.

It enables:

- Checking real-time availability for dental appointments with appointment type matching
- Booking appointments with automatic new/existing patient detection
- Cancelling appointments with intelligent reason code selection
- Looking up existing patients and their upcoming appointments
- Multi-location dental office support with LLM-powered location matching

This integration is designed for **dental clinic administrators and front-desk teams** who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

### 1.2 Use Cases

- When a patient asks about available slots → Check Denticon availability with appointment type resolution and return open time slots
- When a patient wants to book an appointment → Search/create patient record and schedule appointment in Denticon
- When a patient wants to cancel an appointment → Cancel with automatically matched cancellation reason code
- When a clinic has multiple locations → Agent uses LLM to match patient's preferred location from natural language
- When a patient asks about their upcoming visits → Look up patient by name/DOB and retrieve future appointments
- When a new patient books → Automatically detect new vs existing patient and handle accordingly

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | ✅ | With LLM-based appointment type matching |
| Book Appointment | ✅ | Auto-detects new vs existing patient |
| Cancel Appointment | ✅ | With LLM-based cancellation reason selection |
| Patient Lookup | ✅ | By name + date of birth |
| Upcoming Appointments | ✅ | Future appointments filtered by timezone |
| Multi-Location Support | ✅ | LLM-powered location matching (Gemini) |
| Single-Location Mode | ✅ | Simplified setup with pre-selected location |
| Timezone Handling | ✅ | Converts slots to clinic's local timezone |
| OAuth 2.0 Token Management | ✅ | Auto-refresh with retry logic (up to 3 attempts) |

| | | |
|---|---|---|
| Reschedule Appointment | ❌ | Cancel + rebook as workaround |
| Historical Data Import | ❌ | Only events after activation |

## 2. Requirements

Before installation:

- Active **Denticon** (PlanetDDS) account with API access
- **OAuth 2.0 Client ID** and **Client Secret**
- **API Subscription Key** (PDDS-Subscription-Key)
- Denticon API base URL (default: `https://api.planetdds.com/denticon`)

## 3. How to Setup

### 3.1 Step 1 — Generate Credentials in Denticon

1. Log in to your **Denticon** / PlanetDDS account
2. Navigate to the API management or developer section
3. Create an **OAuth 2.0 application** to obtain:
    - **Client ID**
    - **Client Secret**
4. Obtain your **PDDS-Subscription-Key** from the API portal
5. Store all credentials securely

### 3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `denticon_client_id` | ✅ | OAuth 2.0 Client ID |
| `denticon_client_secret` | ✅ | OAuth 2.0 Client Secret |
| `denticon_subscription_key` | ✅ | API Subscription Key (PDDS-Subscription-Key) |
| `denticon_base_url` | ❌ | API base URL (default: `https://api.planetdds.com/denticon`) |
| `denticon_multilocation` | ❌ | Enable multi-location support (default: `True`) |
| `denticon_enable_booking` | ❌ | Enable booking capability (default: `True`) |
| `denticon_enable_cancellation` | ❌ | Enable cancellation capability (default: `True`) |
| `denticon_enable_slot_check` | ❌ | Enable availability check (default: `True`) |
| `denticon_check_existing_client` | ❌ | Enable patient lookup (default: `True`) |
| `denticon_enable_client_history` | ❌ | Enable appointment history (default: `False`) |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:

- Authenticates via OAuth 2.0 (client credentials grant)
- Fetches clinic locations from Denticon
- Fetches available providers
- Fetches cancellation reason codes
- Configures booking tools for the agent

## 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

### 4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Denticon

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Check Availability** | `get_available_slots` | When the agent needs to look up open slots |
| **Book Appointment** | `book_slot` | When the agent confirms a booking |
| **Cancel Appointment** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Check Upcoming Visits** | `check_upcoming_appointments` | When the agent looks up a patient's appointments |
| **Session Start** | `session_started` | Preloads offices context for the session |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |

**Available Actions**

- **Get Production Types** — Fetches appointment types for a specific office location
- **Search Available Slots** — Queries Denticon for open appointment times with matched production type
- **Find Patient** — Searches existing patients by name and date of birth
- **Create Appointment** — Books an appointment slot (supports both new and existing patients)
- **Cancel Appointment** — Cancels an existing appointment with auto-selected reason code
- **Get Patient Appointments** — Retrieves upcoming appointments for an existing patient

### 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Denticon Integration
    participant T as Token Service
    participant API as Denticon API
```

```
Note over I,T: Setup (on publish)
A->>I: project_publish_finish
I->>T: POST /connect/token (OAuth 2.0)
T-->>I: access_token
I->>API: GET /offices
API-->>I: Offices list
I->>API: GET /providers
API-->>I: Providers list
I->>API: GET /cancellation-codes
API-->>I: Cancellation reasons
I->>A: Booking tools registered

Note over U,API: Availability Check
U->>A: "Any slots for cleaning on Monday?"
A->>I: get_available_slots (date, location, visit_reason)
I->>I: LLM location matching (multilocation)
I->>API: GET /scheduler-production-types
API-->>I: Production types
I->>I: LLM appointment type matching
I->>API: GET /available-slots (officeId, productionTypeId, date)
API-->>I: Available slots
I->>I: Convert to local timezone
I->>A: result_success (availability[])
A->>U: "Here are the available times..."

Note over U,API: Booking
U->>A: "Book 10:00 AM please"
A->>I: book_slot (customerInfo, dateTime, location)
I->>I: Resolve slot from cache
I->>API: GET /patients (officeId, name, DOB)
API-->>I: Patient results
alt Patient not found
    Note over I: isNewPatient = true
end
I->>API: POST /appointments (slot, patient, isNewPatient)
API-->>I: appointment_id
I->>A: result_success (booking_id)
A->>U: "Your appointment is confirmed!"

Note over U,API: Cancellation
U->>A: "Cancel my appointment"
A->>I: convo_agent_cancel_booking (booking_id, notes)
I->>I: LLM cancellation reason matching
I->>API: POST /appointments/(id)/cancel (reasonCode)
API-->>I: Confirmation
I->>A: result_success (cancelled)
A->>U: "Your appointment has been cancelled"

Note over U,API: Patient Lookup
U->>A: "Do I have any upcoming visits?"
A->>I: check_upcoming_appointments
I->>I: LLM extract patient data
```

```
    I->>API: GET /patients (name, DOB)
    API-->>I: Patient match
    I->>API: GET /appointments (patientId)
    API-->>I: Appointments
    I->>I: Filter future only
    I->>A: result_success (appointments[])
    A->>U: "You have a visit on..."
```

**AvailabilityFlow**

```
flowchart TD
    E["get_available_slots"] --> EX["Extract payload:<br>date, location,
visit_reason"]
    EX --> ML{"Multilocation?"}
    ML -->|"Yes"| LLM1["LLM match location<br>(Gemini)"]
    LLM1 --> FOUND{"Location<br>matched?"}
    FOUND -->|"No"| FB["Fallback:<br>ask user"]
    FOUND -->|"Yes"| STORE["Store location_id<br>in persona"]
    ML -->|"No"| PRE["Use pre-selected<br>location"]
    PRE --> STORE
    STORE --> PT["GET /scheduler-production-types<br>{officeId}"]
    PT --> LLM2["LLM match visit_reason<br>to production type"]
    LLM2 --> CHK{"Date<br>provided?"}
    CHK -->|"No"| FB
    CHK -->|"Yes"| API["GET /available-slots<br>{officeId, productionTypeId,
<br>startDate, providerId}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| TZ["Convert slots<br>to local timezone"]
    TZ --> CACHE["Cache slots<br>in persona"]
    CACHE --> OUT["result_success<br>{availability[]}"]
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> EX["Extract customerInfo:<br>name, phone, email, DOB"]
    EX --> LOC["Resolve location_id"]
    LOC --> SLOT["Resolve slot<br>from cache"]
    SLOT --> SEARCH["GET /patients<br>{officeId, name, DOB}"]
    SEARCH --> OK1{"HTTP 200?"}
    OK1 -->|"No"| ERR["ResultError"]
    OK1 -->|"Yes"| MATCH{"Patient found?<br>(name + DOB)"}
    MATCH -->|"Yes"| EXIST["isNewPatient = false<br>Use patient_id"]
    MATCH -->|"No"| NEW["isNewPatient = true"]
    EXIST --> BOOK
    NEW --> BOOK
    BOOK["POST /appointments<br>{patient, slot, officeId,<br>providerId,
operatoryId,<br>productionTypeId}"] --> OK2{"HTTP 201?"}
    OK2 -->|"No"| ERR
```

```
    OK2 -->|"Yes"| CLEAR["Clear slots cache"]
    CLEAR --> OUT["result_success<br>{appointment_id}"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> VAL{"booking_id<br>present?"}
    VAL -->|"No"| FB["Fallback:<br>cancellation failed"]
    VAL -->|"Yes"| LLM["LLM match<br>cancellation reason<br>from notes"]
    LLM --> API["POST /appointments/{id}/cancel<br>{cancellationCodeId, note}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

**ExistingClientFlow**

```
flowchart TD
    E["check_upcoming_appointments"] --> CHK{"Feature<br>enabled?"}
    CHK -->|"No"| SKIP["Skip"]
    CHK -->|"Yes"| LLM["LLM extract<br>firstName, lastName,<br>birthDate"]
    LLM --> SEARCH["GET /patients<br>{officeId, name, DOB}"]
    SEARCH --> OK1{"HTTP 200?"}
    OK1 -->|"No"| ERR["ResultError"]
    OK1 -->|"Yes"| MATCH{"Patient<br>found?"}
    MATCH -->|"No"| NOTFOUND["result_error:<br>patient not found"]
    MATCH -->|"Yes"| APPTS["GET /appointments<br>{officeId, patientId}"]
    APPTS --> OK2{"HTTP 200?"}
    OK2 -->|"No"| ERR
    OK2 -->|"Yes"| FILTER["Filter future<br>appointments only"]
    FILTER --> OUT["result_success<br>{appointments[]}"]
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent. API responses are logged via the integration's HTTP connector. Cancellation can also be tested via the `test_integration` webhook event.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify logs show successful OAuth authentication, office fetch, and cancellation codes fetch
2. **Availability**: Ask the agent "What slots are available for a cleaning on Monday?" — verify slots returned with correct appointment type
3. **Booking**: Complete a booking conversation — verify appointment appears in Denticon dashboard
4. **Cancellation**: Request cancellation of a booked appointment — verify status changed in Denticon with correct reason code
5. **Patient Lookup**: Ask "Do I have any upcoming visits?" — verify patient is found and future appointments returned

**If no action occurs:**

- Ensure OAuth credentials are set correctly ( `denticon_client_id` , `denticon_client_secret` , `denticon_subscription_key` )
- Verify `denticon_base_url` points to the correct endpoint
- Check that OAuth token was obtained (look for `denticon_access_token` in attributes)
- Confirm the integration was re-published after configuration changes
- Check that the relevant feature flags are enabled
- In multi-location mode: verify `denticon_offices` was populated during setup

> *Note: This integration performs real operations in Denticon. Appointments created or cancelled through the agent are reflected in the live Denticon system.*

## 5. FAQ

**Q: Does the integration import historical appointments?** A: No. Only appointments created or managed after activation are processed. Patient lookup shows future appointments only.

**Q: Can I connect multiple Denticon accounts?** A: Each integration instance supports one set of OAuth credentials. For multiple accounts, create separate integration instances.

**Q: What happens if a patient already exists in Denticon?** A: The booking flow searches for existing patients by first name, last name, and date of birth. If a match is found, the existing patient record is used with `isNewPatient=false` . If not, the appointment is created with `isNewPatient=true` .

**Q: How does appointment type matching work?** A: When a patient describes their visit reason (e.g., "teeth cleaning", "checkup"), an LLM (Gemini) matches it to available production types configured for the selected office in Denticon. For generic requests, it defaults to the general exam option.

**Q: How does multi-location mode work?** A: When enabled ( `denticon_multilocation=True` ), the agent uses an LLM to match the patient's natural language location description to available clinic offices. The matched location is cached for the session.

**Q: What happens if the OAuth token expires?** A: The integration automatically detects HTTP 401 responses and refreshes the token using the client credentials grant. It retries the failed request up to 3 times.

**Q: How are cancellation reasons selected?** A: During setup, all available cancellation codes are fetched from Denticon. When a cancellation occurs, an LLM analyzes the cancellation notes and selects the most appropriate reason code. If notes are empty, a neutral default reason is used.